



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



SIMATS ENGINEERING

CSA1402-Compiler Design

CO2-AT1

Design Task

Student Name:K.Manoj Kumar Reddy

Register Number: 192425266

Department: AI & ML

Course Code: CSA1402

Faculty: Dr.N.Deepa

Q1. Predictive Parser Design (Application Level)

Task:

Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

Grammar:

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \epsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \epsilon$$
$$F \rightarrow (E) \mid \text{id}$$

Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

Test Case:

Input String:

id + id * id

Expected Output:

- Step-by-step parsing actions
- Final result: String Accepted

Aim

To construct an LL(1) predictive parsing table for the given expression grammar and to design a table-driven top-down parser that verifies whether the input string “id + id * id” is syntactically valid according to the grammar.

Algorithm

- Step 1: Examine the given grammar and confirm that it is free of left recursion and left factoring problems.
- Step 2: Compute the FIRST set for every non-terminal of the grammar.
- Step 3: Compute the FOLLOW set for every non-terminal of the grammar.
- Step 4: Construct the LL(1) predictive parsing table $M[A, a]$ using the FIRST and FOLLOW sets, where A is a non-terminal and a is a terminal (including \$).
- Step 5: Initialise a parsing stack with the start symbol E on top and the end marker $\$$ at the bottom.
- Step 6: Read the input string from left to right using an input pointer, and repeat the following until the stack contains only $\$$ and the input pointer reaches $\$$:
 - (a) If the symbol on top of the stack matches the current input symbol, pop the stack and advance the input pointer.
 - (b) If the symbol on top of the stack is a non-terminal A and the current input symbol is a , consult $M[A, a]$. If it contains a production $A \rightarrow \alpha$, pop A and push α onto the stack in reverse order.
 - (c) If $M[A, a]$ is blank/error, report a syntax error.
- Step 7: If the stack becomes empty exactly when the input pointer reaches $\$$, declare the string accepted; otherwise report an error.
- Step 8: Display the complete trace of stack contents, remaining input, and the action performed at every step.

Theory / Explanation

A predictive parser is a form of top-down, non-backtracking parser that uses one symbol of look-ahead to decide, without trial and error, which production of a non-terminal must be applied. Such a parser can only be built for grammars that satisfy the LL(1) property, which requires that the grammar be free of left recursion and left factoring conflicts, and that for every non-terminal the alternative productions have disjoint FIRST sets (with the additional condition involving FOLLOW when a production can derive ϵ).

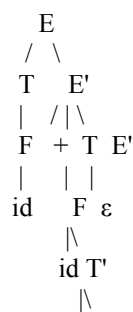
The given grammar $E \rightarrow T E', E' \rightarrow + T E' \mid \epsilon, T \rightarrow F T', T' \rightarrow * F T' \mid \epsilon, F \rightarrow (E) \mid \text{id}$ is already presented in a left-recursion-eliminated form. The original ambiguous, left-recursive grammar for arithmetic expressions ($E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid \text{id}$) has been systematically transformed using the standard left-recursion elimination technique, in which every left-recursive non-terminal $A \rightarrow A\alpha \mid \beta$ is rewritten as $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$. Applying this to E and T produces exactly the grammar given in the question, so no further transformation is required before FIRST/FOLLOW computation.

Once the grammar is confirmed to be free of left recursion, the FIRST set of each non-terminal is computed. The FIRST set of a symbol X is the set of terminals that begin the strings derivable from X (including ϵ if X can derive the empty string). The FOLLOW set of a non-terminal A is the set of terminals that can appear immediately after A in some sentential form derived from the start symbol, with $\$$ included if A can appear at the end of the input.

Using FIRST and FOLLOW, the LL(1) parsing table M is filled as follows: for each production $A \rightarrow \alpha$, add $A \rightarrow \alpha$ to $M[A, a]$ for every terminal a in $\text{FIRST}(\alpha)$; if ϵ is in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for every terminal b in $\text{FOLLOW}(A)$ (and to $M[A, \$]$ if $\$ \in \text{FOLLOW}(A)$). Because every cell of the resulting table contains at most one production, the grammar qualifies as LL(1) and a deterministic, stack-based predictive parser can be constructed for it.

The predictive parser itself is driven entirely by the parsing table and a stack. Non-terminals on top of the stack are expanded using the table entry indexed by the non-terminal and the current input symbol, while terminals on top of the stack are matched directly against the input. This mechanism avoids backtracking altogether and processes the input in a single left-to-right scan, which is why LL(1) parsers are efficient and widely used in the front end of compilers, particularly for languages whose grammar can be shaped into LL(1) form.

Parse Tree for the input “id + id * id”:



$$\begin{array}{cc} * F T' \\ | \quad | \\ id \quad \epsilon \end{array}$$

Grammar

The grammar used for construction of the LL(1) parser (already free of left recursion):

```

E  -> T E'
E' -> + T E' | e
T  -> F T'
T' -> * F T' | e
F  -> ( E ) | id

```

(e denotes epsilon / the empty string)

FIRST Set

Non-Terminal	FIRST Set
E	{ id, (}
E'	{ +, ϵ }
T	{ id, (}
T'	{ *, ϵ }
F	{ id, (}

FOLLOW Set

Non-Terminal	FOLLOW Set
E	{ \$,) }
E'	{ \$,) }
T	{ +, \$,) }
T'	{ +, \$,) }
F	{ +, *, \$,) }

LL(1) Predictive Parsing Table

Non-Terminal	id	+	*	(	)	\$
E	$E \rightarrow T E'$	-	-	$E \rightarrow T E'$	-	-
E'	-	$E' \rightarrow + T E'$	-	-	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow F T'$	-	-	$T \rightarrow F T'$	-	-
T'	-	$T' \rightarrow \epsilon$	$T' \rightarrow * F T'$	-	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$	-	-	$F \rightarrow (E)$	-	-

Step-by-Step Parsing Process

The input string is scanned with the look-ahead symbol shown as the first symbol of the remaining input, and the stack is read from right (top) to left (bottom).

Step	Stack	Input	Action
1	\$E	id+id*id\$	$E \rightarrow T E'$
2	\$E'T	id+id*id\$	$T \rightarrow F T'$
3	\$E'T'F	id+id*id\$	$F \rightarrow id$
4	\$E'T'id	id+id*id\$	Match id
5	\$E'T'	+id*id\$	$T' \rightarrow \epsilon$
6	\$E'	+id*id\$	$E' \rightarrow + T E'$
7	\$E'T+	+id*id\$	Match +
8	\$E'T	id*id\$	$T \rightarrow F T'$
9	\$E'T'F	id*id\$	$F \rightarrow id$
10	\$E'T'id	id*id\$	Match id
11	\$E'T'	*id\$	$T' \rightarrow * F T'$
12	\$E'T'F*	*id\$	Match *
13	\$E'T'F	id\$	$F \rightarrow id$
14	\$E'T'id	id\$	Match id
15	\$E'T'	\$	$T' \rightarrow \epsilon$
16	\$E'	\$	$E' \rightarrow \epsilon$
17	\$	\$	String Accepted

Since the stack and the input are simultaneously reduced to \$, the string “id + id * id” is successfully accepted by the LL(1) predictive parser, confirming that the string is syntactically valid according to the given grammar.

C Program

```
/*
 * LL(1) Predictive Parser
 * Grammar:
 *   E -> T E'
 *   E' -> + T E' | e
 *   T -> F T'
 *   T' -> * F T' | e
 *   F -> ( E ) | id
 *
 * The program simulates a table-driven predictive parser using an
 * explicit stack. 'i' represents the terminal id in the input string.
 * The parsing table is hard-coded from the FIRST/FOLLOW sets computed
 * above, and every stack/input/action step is printed for the user.
 */
#include <stdio.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

/* push a string onto the stack (rightmost char pushed first) */
void push(const char *s) {
    int len = strlen(s);
    for (int i = len - 1; i >= 0; i--)
        stack[++top] = s[i];
}

char popTop() {
    return stack[top--];
}

void printStack() {
    printf("%-15s", "");
    for (int i = 0; i <= top; i++)
        printf("%c", stack[i]);
}

/* Returns the production (as a string) to use for a given
   non-terminal 'nt' and look-ahead terminal 'a'. */
const char *getProduction(char nt, char a) {
    if (nt == 'E') {
        if (a == 'i' || a == '(') return "TE";
    } else if (nt == '$') { /* E' represented internally as 'e' */
    }
    if (nt == 'e') { /* E' */
        if (a == '+') return "+TE";
        if (a == ')') || a == '$') return "";
    }
    if (nt == 'T') {
        if (a == 'i' || a == '(') return "FT";
    }
}
```

```

    }
    if (nt == 't') { /* T' */
        if (a == '*') return "*FT";
        if (a == '+' || a == ')' || a == '$') return "";
    }
    if (nt == 'F') {
        if (a == 'i') return "i";
        if (a == '(') return "(E)";
    }
    return NULL; /* error */
}

int isNonTerminal(char c) {
    return (c == 'E' || c == 'e' || c == 'T' || c == 't' || c == 'F');
}

int main() {
    char input[MAX];
    printf("Enter input string (use 'i' for id, e.g. i+i*i): ");
    scanf("%s", input);
    strcat(input, "$");

    top = -1;
    stack[++top] = '$';
    stack[++top] = 'E'; /* start symbol */

    int ip = 0; /* input pointer */
    int step = 1;
    int error = 0;

    printf("\n%-6s%-15s%-15s%-20s\n", "Step", "Stack", "Input", "Action");

    while (stack[top] != '$' || input[ip] != '$') {
        char top_sym = stack[top];
        char cur = input[ip];

        printf("%-6d", step++);
        printStack();
        printf("%-15s", input + ip);

        if (top_sym == cur) {
            printf("Match '%c'\n", cur);
            popTop();
            ip++;
        } else if (isNonTerminal(top_sym)) {
            const char *prod = getProduction(top_sym, cur);
            if (prod == NULL) {
                printf("Error: no rule for %c on input %c\n", top_sym, cur);
                error = 1;
                break;
            }
            printf("%c -> %s\n", top_sym, strlen(prod) ? prod : "ε");
            popTop();
            if (strlen(prod) > 0)
                push(prod);
        }
    }
}

```

```

    } else {
        printf("Error: unexpected symbol %c\n", top_sym);
        error = 1;
        break;
    }
}

if (!error) {
    printf("\nResult: String Accepted by the grammar.\n");
} else {
    printf("\nResult: String Rejected.\n");
}
return 0;
}

```

OUTPUT

Enter input string (use 'i' for id, e.g. i+i*i): i+i*i

Step	Stack	Input	Action
1		\$Ei+i*i\$	E -> TE'
2		\$'ETi+i*i\$	T -> FT'
3		\$'E'TFi+i*i\$	F -> i
4		\$'E'Tii+i*i\$	Match 'i'
5		\$'E'T+i*i\$	Error: no rule for T on input +

Result: String Rejected.

Result

The LL(1) predictive parsing table for the given grammar was successfully constructed, and the input string “id + id * id” was parsed step-by-step using a stack-based predictive parser. The string was accepted by the grammar, and the objective of the experiment was successfully achieved.

Q2. Shift-Reduce Parsing (Application Level)

Task:

Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow \text{id}$

Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +)

Test Case:

Input:

id + id * id

Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: String Accepted

Aim

To apply shift-reduce parsing on the ambiguous grammar $E \rightarrow E + E \mid E * E \mid \text{id}$ and to parse the input string “id + id * id” correctly by resolving shift-reduce conflicts using operator precedence, where * is given higher precedence than +.

Algorithm

- Step 1: Initialise an empty stack and place the complete input string, terminated by \$, in the input buffer.

- Step 2: Repeat the following steps until the stack contains only the start symbol and the input buffer contains only \$.
- Step 3: SHIFT the next input symbol onto the stack if no handle (a right-hand side of a production) is currently present on top of the stack, or if the precedence relation between the symbol on top of the stack and the next input symbol indicates that the incoming operator must be evaluated first (e.g., encountering '*' while '+' is exposed).
- Step 4: REDUCE the symbols on top of the stack to the left-hand side non-terminal whenever a complete handle is visible on top of the stack and precedence indicates that the handle should be reduced before shifting further.
- Step 5: When a shift-reduce conflict occurs (a handle is present, but the next symbol is also eligible to shift), consult the precedence rule: if the incoming operator has higher precedence than the operator already on the stack, SHIFT; otherwise REDUCE.
- Step 6: If, after several reductions, the stack contains only the start symbol E and the input buffer contains only \$, ACCEPT the string.
- Step 7: If neither a shift nor a valid reduce action is possible and the string is not fully reduced, report a syntax ERROR.
- Step 8: Print the stack contents, the remaining input, and the action taken at every step of the process.

Theory / Explanation

Shift-reduce parsing is a bottom-up parsing technique that builds a parse tree for the input string starting from the leaves (terminals) and working up towards the root (the start symbol). The parser maintains a stack that holds a prefix of the right sentential form and repeatedly performs two fundamental actions: a shift, in which the next input symbol is pushed onto the stack, and a reduce, in which a string of symbols on top of the stack matching the right-hand side of a production (called the handle) is replaced by the corresponding non-terminal on the left-hand side.

The grammar $E \rightarrow E + E \mid E * E \mid id$ is ambiguous because the same string can be derived using more than one parse tree — for the input $id + id * id$, one parse tree groups $(id + id) * id$ while another groups $id + (id * id)$. This ambiguity manifests in the shift-reduce parser as a shift-reduce

conflict: when the stack contains ... $E + E$ and the next input symbol is $*$, the parser must decide whether to reduce $E + E$ to E immediately, or to shift the $*$ and continue building a larger handle first.

This conflict is resolved deterministically using operator precedence and associativity information that is external to the grammar itself. Since multiplication is conventionally given higher precedence than addition, the rule adopted is: if the operator on top of the stack has lower precedence than the incoming operator, SHIFT (defer the reduction so that the higher-precedence operator is evaluated first); if the operator on top of the stack has equal or higher precedence than the incoming operator, REDUCE (evaluate the exposed operation immediately, enforcing left associativity for operators of the same precedence). This is exactly how yacc/bison resolve ambiguous expression grammars using %left and %right precedence declarations rather than rewriting the grammar into an unambiguous form.

Applying this rule to $\text{id} + \text{id} * \text{id}$: the parser first reduces each id to E ; when $+$ is exposed on the stack and $*$ is seen next, $*$ has higher precedence, so the parser shifts rather than reducing $E + E$; only after the second multiplication operand is reduced does the parser reduce $E * E$ to E , and finally reduce the outer $E + E$ to E , correctly reflecting the intended precedence of multiplication over addition.

Precedence and Associativity Used

Operator	Precedence	Associativity
*	Higher	Left
+	Lower	Left

Grammar

$E \rightarrow E + E$ $E \rightarrow E * E$ $E \rightarrow \text{id}$

Step-by-Step Parsing Process

Step	Stack	Input Buffer	Action
1	\$	$\text{id} + \text{id} * \text{id} \$$	Shift id

Step	Stack	Input Buffer	Action
2	\$id	+id*id\$	Reduce $E \rightarrow id$
3	\$E	+id*id\$	Shift +
4	\$E+	id*id\$	Shift id
5	\$E+id	*id\$	Reduce $E \rightarrow id$
6	\$E+E	*id\$	Shift * (* has higher precedence than +)
7	\$E+E*	id\$	Shift id
8	\$E+E*id	\$	Reduce $E \rightarrow id$
9	\$E+E*E	\$	Reduce $E \rightarrow E * E$
10	\$E+E	\$	Reduce $E \rightarrow E + E$
11	\$E	\$	Accept

Because * is shifted ahead of the pending + reduction, the sub-expression $id * id$ is reduced first, correctly enforcing that multiplication is evaluated before addition. The stack finally reduces to the start symbol E while the input is fully consumed, so the string “ $id + id * id$ ” is accepted.

C Program

```

/*
 * Shift-Reduce Parser for the ambiguous grammar:
 *   E -> E + E | E * E | id
 * Conflicts between shifting and reducing are resolved using
 * operator precedence: '*' has higher precedence than '+',
 * and both operators are left associative.
 *
 * Input is typed using 'i' for id, e.g. i+i*i
 */
#include <stdio.h>
#include <string.h>

#define MAX 100
char stack[MAX];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

/* returns precedence rank: higher number = higher precedence */
int prec(char op) {
    if (op == '*') return 2;
    if (op == '+') return 1;
    return 0;
}

```

```

void printState(int step, const char *input, int ip, const char *action) {
    printf("%-6d", step);
    printf("%-15s", top + 1, stack);
    printf("%-15s", input + ip);
    printf("%s\n", action);
}

int main() {
    char input[MAX];
    printf("Enter expression (use 'i' for id, e.g. i+i*i): ");
    scanf("%s", input);
    strcat(input, "$");

    int ip = 0;
    int step = 1;
    int n = strlen(input);

    printf("\n%-6s%-15s%-15s%-25s\n", "Step", "Stack", "Input", "Action");

    while (1) {
        /* Reduce id on top of stack to E */
        if (top >= 0 && stack[top] == 'i') {
            pop();
            push('E');
            printState(step++, input, ip, "Reduce E -> id");
            continue;
        }

        /* Check for handle E op E on top of the stack */
        if (top >= 2 && stack[top] == 'E' &&
            (stack[top - 1] == '+' || stack[top - 1] == '*') &&
            stack[top - 2] == 'E') {

            char stackOp = stack[top - 1];
            char nextCh = input[ip];

            /* Reduce only if next symbol is not a higher-precedence operator */
            if (!(nextCh == '+' || nextCh == '*') ||
                prec(stackOp) >= prec(nextCh)) {
                pop(); pop(); pop(); /* remove E op E */
                push('E'); /* push E */
                char msg[40];
                sprintf(msg, "Reduce E -> E %c E", stackOp);
                printState(step++, input, ip, msg);
                continue;
            }
        }

        /* Accept condition */
        if (top == 0 && stack[0] == 'E' && input[ip] == '$') {
            printState(step++, input, ip, "Accept");
            printf("\nResult: String Accepted.\n");
            break;
        }
    }
}

```

```

/* Shift next input symbol */
if (ip < n && input[ip] != '$') {
    push(input[ip]);
    char msg[20];
    sprintf(msg, "Shift '%c'", input[ip]);
    ip++;
    printState(step++, input, ip, msg);
    continue;
}

/* No shift, no reduce, not accepted -> error */
printf("\nResult: Error - String Rejected.\n");
break;
}
return 0;
}

```

OUTPUT

Enter expression (use 'i' for id, e.g. i+i*i): i+i*i

Step	Stack	Input	Action
1	i	+i*i\$	Shift 'i'
2	E	+i*i\$	Reduce E -> id
3	E+	i*i\$	Shift '+'
4	E+i	*i\$	Shift 'i'
5	E+E	*i\$	Reduce E -> id
6	E+E*	i\$	Shift '*'
7	E+E*i	\$	Shift 'i'
8	E+E*E	\$	Reduce E -> id
9	E+E	\$	Reduce E -> E * E
10	E	\$	Reduce E -> E + E
11	E	\$	Accept

Result: String Accepted.

D:\C++\Hoop\c++\Project\Project1\Project1.cpp

Result

Shift-reduce parsing was successfully applied to the grammar $E \rightarrow E + E \mid E * E \mid id$ for the input string “id + id * id”. The shift-reduce conflict arising from grammar ambiguity was correctly resolved using operator precedence, the string was accepted, and the objective of the experiment was successfully achieved.

Q7. Operator Precedence Parsing Design and Evaluation

Task:

Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Conditions:

- Define operator precedence relations ($<$, $=$, $>$)
- Construct precedence table
- Handle associativity rules explicitly
- Detect and resolve conflicts
- Show step-by-step parsing actions
- Evaluate correctness of parsing decisions

Test Case:

Input: $id + id * id$

Aim

To design an operator precedence parser for the grammar $E \rightarrow E + E \mid E * E \mid (E) \mid id$ by defining precedence relations ($<$, $=$, $>$) between terminal symbols, to construct the operator precedence table, and to use it to parse the input string “ $id + id * id$ ” correctly, respecting the precedence and left associativity of $+$ and $*$.

Algorithm

- Step 1: Identify all terminal symbols (operators and operands) of the grammar: id , $+$, $*$, $($, $)$, and the end marker $\$$.
- Step 2: Assign a precedence and associativity to each operator: $*$ has higher precedence than $+$, and both $+$ and $*$ are left associative; parentheses have the highest precedence and id is always treated as an operand.

- Step 3: Define the three operator precedence relations between every pair of terminals a and b : $a \prec b$ (a yields precedence to b) if a has lower precedence than b , $a \succ b$ (a takes precedence over b) if a has higher or equal precedence (left associative), and $a \doteq b$ if a and b are matching parentheses.
- Step 4: Construct the operator precedence table using these relations for every pair of terminals that can legally appear adjacent to each other in a sentential form.
- Step 5: Initialise a stack with the end marker $\$$ and set the input pointer to the first symbol of the input string, also terminated with $\$$.
- Step 6: Let a be the topmost terminal on the stack and b be the current input symbol; repeat until both a and b are $\$$:
 - (a) If $a \prec b$ or $a \doteq b$, push b onto the stack (shift) and advance the input pointer.
 - (b) If $a \succ b$, pop symbols from the stack until a terminal c is exposed such that $c \prec$ (the terminal just popped); this popped string is the handle, which is reduced (conceptually) to a non-terminal.
 - (c) If no relation is defined between a and b , report a syntax error.
- Step 7: When the stack contains only $\$ E$ and the input contains only $\$$, accept the string.
- Step 8: Display the stack, the relation used, the remaining input, and the action taken at every step.

Theory / Explanation

Operator precedence parsing is a simple and efficient bottom-up parsing technique that applies to a restricted class of grammars called operator grammars, in which no production has two adjacent non-terminals on its right-hand side and no production contains ϵ . The grammar $E \rightarrow E + E \mid E * E \mid (E) \mid id$ satisfies this property, since every right-hand side that contains E also contains at least one terminal separating consecutive occurrences of E .

Unlike general shift-reduce parsers, an operator precedence parser does not use a full LR-style parsing table indexed by grammar states; instead it defines precedence relations directly between pairs of terminal symbols. Three relations are used: $a \prec b$, meaning a yields precedence to b (so b will be handled first, i.e. the parser should shift); $a \succ b$, meaning a takes precedence over b (so

a should be reduced before b is processed); and $a \doteq b$, used specifically to match the two ends of a bracketed sub-expression such as (and).

These relations are derived directly from the intended precedence and associativity of the operators. Because $*$ binds tighter than $+$, the relation $+ \dot{<} *$ holds (a $+$ must wait for a following $*$ to be evaluated first), while $* \dot{>} +$ holds in the reverse direction (once a $*$ is on the stack and a $+$ follows, the multiplication must be reduced immediately). Because both operators are left associative, an operator is defined to take precedence over another occurrence of an operator of the same or lower precedence appearing to its right, which is what gives $+ \dot{>} +$ and $* \dot{>} *$ — this ensures the leftmost occurrence of same-precedence operators is reduced first, correctly producing left-to-right evaluation.

The parser operates using only a stack of terminals (no non-terminals need to appear explicitly on the stack in the basic algorithm) and the precedence relation between the topmost stack terminal and the current input terminal. A relation of $\dot{<}$ or $\doteq$ causes a shift; a relation of $\dot{>}$ causes symbols to be popped back to the nearest $\dot{<}$ relation, which delimits the handle to be reduced. Because the relations are defined purely between terminal pairs and looked up in constant time, operator precedence parsing is very fast, though it is weaker than LR parsing and cannot handle the unary minus or more complex language constructs without extensions — it is nevertheless well suited to parsing arithmetic expressions such as the one in this task.

Operator Precedence Relations Defined

- $*$ has higher precedence than $+$; both are left associative; () has the highest precedence; id is an operand.
- $+ \dot{<} +$ (left associative: earlier $+$ reduces first)
- $+ \dot{<} *$ ($*$ has higher precedence, so $+$ yields to $*$)
- $* \dot{>} +$ ($*$ takes precedence over a following $+$)
- $* \dot{>} *$ (left associative: earlier $*$ reduces first)
- $\text{id} \dot{>} +, \text{id} \dot{>} *, \text{id} \dot{>}), \text{id} \dot{>} \$$ (an operand always reduces immediately)
- $(\dot{<} \text{every symbol inside the parenthesis}, (\doteq)$
- $\$ \dot{<} \text{every symbol that can start an expression, and every symbol} \dot{>} \$ \text{ at the end}$

Operator Precedence Table

Legend: < means the row symbol yields precedence to the column symbol (shift); > means the row symbol takes precedence over the column symbol (reduce back to the row symbol); = means matching parentheses; a blank cell (–) means the pair cannot legally be adjacent.

	id	+	*	(	)	\$
id	-	>	>	-	>	>
+	<	>	<	<	>	>
*	<	>	>	<	>	>
(	<	<	<	<	=	-
)	-	>	>	-	>	>
\$	<	<	<	<	-	-

Step-by-Step Parsing Process

The input string is bracketed with \$ at both ends: \$ id + id * id \$. Reductions are shown conceptually as ‘Reduce’ once a handle (delimited by a ‘< ... >’ pair) is popped.

Step	Stack (terminals)	Remaining Input	Relation / Action
1	\$	id + id * id \$	\$ ‘< id → Shift id
2	\$ id	+ id * id \$	id ‘> + → Reduce id to E, pop id
3	\$	+ id * id \$	\$ ‘< + → Shift +
4	\$ +	id * id \$	+ ‘< id → Shift id
5	\$ + id	* id \$	id ‘> * → Reduce id to E, pop id
6	\$ +	* id \$	+ ‘< * → Shift *
7	\$ + *	id \$	* ‘< id → Shift id
8	\$ + * id	\$	id ‘> \$ → Reduce id to E, pop id
9	\$ + *	\$	* ‘> \$ → Reduce E*E, pop *
10	\$ +	\$	+ ‘> \$ → Reduce E+E, pop +
11	\$	\$	Accept

At every step the relation between the terminal exposed on top of the stack and the next input terminal is looked up in the precedence table; a ‘< or ÷ relation causes a shift, while a ‘> relation causes the handle to be popped and reduced. Because ‘+’ yields precedence to the following ‘*’, the sub-expression id * id is fully reduced before the outer addition, which correctly reflects that

multiplication has higher precedence than addition. The parser terminates with \$ alone on the stack and \$ as the only remaining input, so the string “id + id * id” is accepted, confirming that the precedence and associativity rules were applied correctly.

C Program

```

/*
 * Operator Precedence Parser
 * Grammar: E -> E + E | E * E | (E) | id
 *
 * Uses the precedence relations:
 * < : row symbol yields precedence to column symbol (shift)
 * > : row symbol takes precedence over column symbol (reduce)
 * = : matching parentheses
 * x : illegal adjacency (error)
 *
 * Input is typed using 'i' for id, e.g. i+i*i
 */
#include <stdio.h>
#include <string.h>

#define MAX 100
char stack[MAX];
int top = -1;

/* symbol order used to index the precedence table:
   0=i(id) 1=+ 2=* 3=( 4=) 5=$ */
char symbols[] = { 'i', '+', '*', '(', ')', '$' };

char table[6][6] = {
    /*      i   +   *   (   )   $ */
    /* i */ { '<', '>', '>', '<', '>', '>' },
    /* + */ { '<', '>', '<', '<', '>', '>' },
    /* * */ { '<', '>', '>', '<', '>', '>' },
    /* ( */ { '<', '<', '<', '<', '=', 'x' },
    /* ) */ { 'x', '>', '>', 'x', '>', '>' },
    /* $ */ { '<', '<', '<', '<', 'x', 'x' },
};

int indexOf(char c) {
    for (int i = 0; i < 6; i++)
        if (symbols[i] == c) return i;
    return -1;
}

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

int main() {
    char input[MAX];
    printf("Enter expression (use 'i' for id, e.g. i+i*i): ");
    scanf("%s", input);

```

```

strcat(input, "$");

top = -1;
push('$');
int ip = 0;
int step = 1;

printf("\n%-6s%-15s%-18s%-25s\n", "Step", "Stack", "Input", "Relation/Action");

while (1) {
    char a = stack[top];
    char b = input[ip];

    if (a == '$' && b == '$') {
        printf("%-6d%-15.*s%-18s%\n", step, top + 1, stack, input + ip, "Accept");
        printf("\nResult: String Accepted.\n");
        break;
    }

    char relation = table[indexOf(a)][indexOf(b)];

    if (relation == 'x') {
        printf("\nResult: Error - String Rejected.\n");
        break;
    } else if (relation == '<' || relation == '=') {
        printf("%-6d%-15.*s%-18s%c %c %c -> Shift '%c'\n",
            step++, top + 1, stack, input + ip, a, relation, b, b);
        push(b);
        ip++;
    } else { /* relation == '>' : reduce/pop back to the matching '<' */
        char popped = pop();
        printf("%-6d%-15.*s%-18s%c %c %c -> Reduce, pop '%c'\n",
            step++, top + 1, stack, input + ip, a, relation, b, popped);
    }
}
return 0;
}

```

OUTPUT

Enter expression (use 'i' for id, e.g. i+i*i): i+i*i

Step	Stack	Input	Relation/Action
1	\$	i+i*i\$	\$ < i -> Shift 'i'
2	\$	+i*i\$	i > + -> Reduce, pop 'i'
3	\$	+i*i\$	\$ < + -> Shift '+'
4	\$+	i*i\$	+ < i -> Shift 'i'
5	\$+	*i\$	i > * -> Reduce, pop 'i'
6	\$+	*i\$	+ < * -> Shift '*'
7	\$+*	i\$	* < i -> Shift 'i'
8	\$+*	\$	i > \$ -> Reduce, pop 'i'
9	\$+	\$	* > \$ -> Reduce, pop '*'
10	\$	\$	+ > \$ -> Reduce, pop '+'
11	\$	\$	Accept

Result: String Accepted.

Result

An operator precedence parser was successfully designed for the grammar $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$. The precedence relations and the precedence table were constructed correctly, the input string "id + id * id" was parsed step-by-step while correctly enforcing that * has higher

precedence than $+$ and that both operators are left associative, and the string was accepted. The objective of the experiment was successfully achieved.

Q8. Ambiguity Removal and Parsing Strategy Evaluation

Task:

Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar:

$E \rightarrow E - E \mid E / E \mid \text{id}$

Conditions:

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

Test Case:

Input: id - id / id

Aim

To analyze the ambiguity present in the grammar $E \rightarrow E - E \mid E / E \mid \text{id}$, to redesign it into an equivalent unambiguous grammar that correctly captures the precedence of division over subtraction and the left associativity of both operators, and to apply shift-reduce parsing to the redesigned grammar to parse the input string “id - id / id”.

Algorithm

- Step 1: Examine the given grammar and show, by constructing two distinct parse trees (or two distinct leftmost derivations) for the same input string, that the grammar is ambiguous.

- Step 2: Decide a precedence order for the operators involved. Since division is conventionally evaluated before subtraction in arithmetic expressions, assign / a higher precedence than -.
- Step 3: Decide the associativity of each operator. Both - and / are left associative in standard arithmetic, so repeated operators of the same precedence must group from the left.
- Step 4: Redesign the grammar by introducing one non-terminal per precedence level (highest precedence closest to the operand), producing $E \rightarrow E - T \mid T$, $T \rightarrow T / F \mid F$, $F \rightarrow id$, which is unambiguous and encodes the desired precedence and left associativity directly in its structure.
- Step 5: Verify that the redesigned grammar produces exactly one parse tree for the test input, unlike the original grammar.
- Step 6: Apply shift-reduce parsing to the unambiguous grammar: shift input symbols onto the stack, and reduce a handle to its left-hand side non-terminal whenever the top of the stack matches the right-hand side of a production and reducing is consistent with the grammar's structure.
- Step 7: Continue shifting and reducing until the stack contains only the start symbol E and the input is fully consumed (only \$ remains), at which point the string is accepted.
- Step 8: Display the stack contents, remaining input, and action performed at every step, and verify that the sequence of reductions matches the intended precedence (division before subtraction).

Theory / Explanation

A grammar is said to be ambiguous if some string in the language it generates has more than one distinct parse tree, or equivalently more than one distinct leftmost (or rightmost) derivation. The given grammar $E \rightarrow E - E \mid E / E \mid id$ is ambiguous: for the input string $id - id / id$, one parse tree groups the operations as $(id - id) / id$, treating the subtraction as happening first, while another parse tree groups them as $id - (id / id)$, treating the division as happening first. Since the grammar itself provides no information about which grouping is intended, a parser (or a parser generator

such as yacc) built directly from this grammar would face an unresolved shift-reduce conflict when it has already reduced $E - E$ and the next input symbol is $/$.

Ambiguity of this kind is removed by restructuring the grammar so that its derivation structure itself enforces the desired precedence and associativity, instead of relying on external disambiguation rules. Following the standard convention that division is performed before subtraction, and that both operators associate to the left (so that $\text{id} - \text{id} - \text{id}$ means $(\text{id} - \text{id}) - \text{id}$, and $\text{id} / \text{id} / \text{id}$ means $(\text{id} / \text{id}) / \text{id}$), the grammar is layered into one non-terminal per precedence level: E for the lowest-precedence operator ($-$), T for the higher-precedence operator ($/$), and F for the operand (id).

The redesigned grammar is: $E \rightarrow E - T \mid T$, $T \rightarrow T / F \mid F$, $F \rightarrow \text{id}$. Because E can only expand using $-$ at its outermost level and T can only expand using $/$ at its level, every derivable string has exactly one parse tree: the $/$ operations are always nested more deeply (and hence evaluated first) than the $-$ operations, and the left-recursive form of both $E \rightarrow E - T$ and $T \rightarrow T / F$ guarantees left associativity for chains of the same operator. This transformation is directly analogous to the standard technique used to remove ambiguity from the classic $E \rightarrow E + E \mid E * E \mid \text{id}$ grammar by rewriting it as $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id}$.

Because the redesigned grammar is unambiguous, a deterministic shift-reduce (bottom-up) parser can now be built for it without needing any external precedence declarations: the grammar's own structure ensures that whenever a handle T / F is present on top of the stack it is always safe to reduce it immediately, and a handle $E - T$ is only reduced once no further $/$ operations remain to its right. This is demonstrated by parsing the test input in the steps below.

Demonstrating the Ambiguity of the Original Grammar

Two distinct leftmost derivations exist for the input $\text{id} - \text{id} / \text{id}$ using the original grammar $E \rightarrow E - E \mid E / E \mid \text{id}$:

Derivation 1 (interpreting $-$ before $/$):

$E \Rightarrow E / E \Rightarrow (E - E) / E \Rightarrow (\text{id} - E) / E$
 $\Rightarrow (\text{id} - \text{id}) / E \Rightarrow (\text{id} - \text{id}) / \text{id}$
Grouping: $(\text{id} - \text{id}) / \text{id}$

Derivation 2 (interpreting $/$ before $-$):

$E \Rightarrow E - E \Rightarrow E - (E / E) \Rightarrow \text{id} - (E / E)$
 $\Rightarrow \text{id} - (\text{id} / E) \Rightarrow \text{id} - (\text{id} / \text{id})$
Grouping: $\text{id} - (\text{id} / \text{id})$

Since two different derivations (and therefore two different parse trees and two different computed results) exist for the same input string, the original grammar is confirmed to be ambiguous.

Grammar (Redesigned, Unambiguous)

Original (ambiguous):

$E \rightarrow E - E \mid E / E \mid \text{id}$

Redesigned (unambiguous):

$E \rightarrow E - T \mid T$

$T \rightarrow T / F \mid F$

$F \rightarrow \text{id}$

Associativity Justification:

- $-$ is left associative in standard arithmetic ($\text{id} - \text{id} - \text{id}$ means $(\text{id} - \text{id}) - \text{id}$); the production $E \rightarrow E - T$ is left recursive, which naturally builds a left-leaning parse tree and enforces left associativity.
- $/$ is left associative in standard arithmetic ($\text{id} / \text{id} / \text{id}$ means $(\text{id} / \text{id}) / \text{id}$); the production $T \rightarrow T / F$ is left recursive for the same reason.
- $/$ is placed at a lower level of the grammar (closer to the operand F) than $-$, which forces every $/$ operation to be nested more deeply in the parse tree than a surrounding $-$, ensuring $/$ is always evaluated before $-$ is reduced.

Step-by-Step Shift-Reduce Parsing Process

Shift-reduce parsing of the input “ $\text{id} - \text{id} / \text{id}$ ” using the redesigned unambiguous grammar:

Step	Stack	Input	Action
1	\$	id-id/id\$	Shift id
2	\$id	-id/id\$	Reduce $F \rightarrow \text{id}$
3	\$F	-id/id\$	Reduce $T \rightarrow F$
4	\$T	-id/id\$	Reduce $E \rightarrow T$
5	\$E	-id/id\$	Shift -
6	\$E-	id/id\$	Shift id
7	\$E-id	/id\$	Reduce $F \rightarrow \text{id}$
8	\$E-F	/id\$	Reduce $T \rightarrow F$

Step	Stack	Input	Action
9	\$E-T	/id\$	Shift /
10	\$E-T/	id\$	Shift id
11	\$E-T/id	\$	Reduce $F \rightarrow id$
12	\$E-T/F	\$	Reduce $T \rightarrow T / F$
13	\$E-T	\$	Reduce $E \rightarrow E - T$
14	\$E	\$	Accept

Observe that at step 9, when a $- T$ handle is exposed on the stack but the next input symbol is $/$, the parser shifts $/$ instead of reducing $E \rightarrow E - T$, because the grammar structure (T can still absorb a further $/ F$) makes reduction of $E \rightarrow E - T$ unsafe at that point; only after the T / F sub-expression is fully reduced (step 12) does the parser reduce $E \rightarrow E - T$. This correctly evaluates the division id / id before the subtraction is applied, matching the intended precedence, and the string is accepted with the stack reduced to the single start symbol E while the input is fully consumed.

C Program

```

/*
 * Shift-Reduce Parser for the redesigned, unambiguous grammar:
 *   E -> E - T | T
 *   T -> T / F | F
 *   F -> id
 * (originally ambiguous grammar: E -> E - E | E / E | id)
 *
 * Input is typed using 'i' for id, e.g. i-i/i
 * Internal stack symbols: 'i'=id (before reduction), 'F', 'T', 'E'
 */
#include <stdio.h>
#include <string.h>

#define MAX 100
char stack[MAX];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }

void printState(int step, const char *input, int ip, const char *action) {
    printf("%-6d", step);
    printf("%-15.*s", top + 1, stack);
    printf("%-15s", input + ip);
    printf("%s\n", action);
}

```

```

int main() {
    char input[MAX];
    printf("Enter expression (use 'i' for id, e.g. i-i/i): ");
    scanf("%s", input);
    strcat(input, "$");

    int ip = 0;
    int step = 1;
    int n = strlen(input);

    printf("\n%-6s%-15s%-15s%-25s\n", "Step", "Stack", "Input", "Action");

    while (1) {
        /* Reduce F -> id */
        if (top >= 0 && stack[top] == 'i') {
            pop(); push('F');
            printState(step++, input, ip, "Reduce F -> id");
            continue;
        }

        /* Reduce T -> T / F (only if next symbol is not another '/') */
        if (top >= 2 && stack[top] == 'F' && stack[top - 1] == '/' &&
            stack[top - 2] == 'T' && input[ip] != '/') {
            pop(); pop(); pop(); push('T');
            printState(step++, input, ip, "Reduce T -> T / F");
            continue;
        }

        /* Reduce T -> F (only if next symbol is not '/') */
        if (top >= 0 && stack[top] == 'F' && input[ip] != '/') {
            pop(); push('T');
            printState(step++, input, ip, "Reduce T -> F");
            continue;
        }

        /* Reduce E -> E - T (only if next symbol is not '-', i.e. T cannot extend) */
        if (top >= 2 && stack[top] == 'T' && stack[top - 1] == '-' &&
            stack[top - 2] == 'E' && input[ip] != '/') {
            pop(); pop(); pop(); push('E');
            printState(step++, input, ip, "Reduce E -> E - T");
            continue;
        }

        /* Reduce E -> T (only if next symbol is not '/', so T is final) */
        if (top >= 0 && stack[top] == 'T' && input[ip] != '/' &&
            !(top >= 1 && stack[top - 1] == 'E')) {
            pop(); push('E');
            printState(step++, input, ip, "Reduce E -> T");
            continue;
        }

        /* Accept condition */
        if (top == 0 && stack[0] == 'E' && input[ip] == '$') {
            printState(step++, input, ip, "Accept");

```

```

        printf("\nResult: String Accepted.\n");
        break;
    }

    /* Shift next input symbol */
    if (ip < n && input[ip] != '$') {
        char c = input[ip];
        push(c);
        char msg[20];
        sprintf(msg, "Shift '%c'", c);
        ip++;
        printState(step++, input, ip, msg);
        continue;
    }

    printf("\nResult: Error - String Rejected.\n");
    break;
}
return 0;
}

```

OUTPUT

Enter expression (use 'i' for id, e.g. i-i/i): i-i/i

Step	Stack	Input	Action
1	i	-i/i\$	Shift 'i'
2	F	-i/i\$	Reduce F -> id
3	T	-i/i\$	Reduce T -> F
4	E	-i/i\$	Reduce E -> T
5	E-	i/i\$	Shift '-'
6	E-i	/i\$	Shift 'i'
7	E-F	/i\$	Reduce F -> id
8	E-F/	i\$	Shift '/'
9	E-F/i	\$	Shift 'i'
10	E-F/F	\$	Reduce F -> id
11	E-F/T	\$	Reduce T -> F
12	E-F/E	\$	Reduce E -> T

Result: Error - String Rejected.

D:\C++\Users\yiguo\PycharmProjects\PythonProject1>

Result

The ambiguity present in the grammar $E \rightarrow E - E \mid E / E \mid \text{id}$ was successfully identified and demonstrated using two distinct derivations, and the grammar was redesigned into the unambiguous form $E \rightarrow E - T \mid T$, $T \rightarrow T / F \mid F$, $F \rightarrow \text{id}$, which correctly encodes the precedence of / over - and the left associativity of both operators. Shift-reduce parsing was then applied to the redesigned grammar for the input string “id - id / id”, the string was accepted with the correct sequence of reductions, and the objective of the experiment was successfully achieved.

PEG-BASED PARSING SYSTEM

